SCambrian – startup guide

From a clean clone on ORFEO to the first results (current flow, November 2026)

1. The picture

Today the repository is both the **tool** and **one analysis**: you clone it, build its environment once, edit two config files, and run. Every command below is run from the repository root.

```
SCambrian/                      (git clone)
  esmc_config.sh                builds ./.env once  ->  env/ lock files
  configs/query_configs.tsv     WHAT to compare (one row per species or set)
  configs/param_configs.yaml    HOW (one section per pipeline step)
  slurms/run_all.sh             submits steps 01 -> 02 -> 03 -> supp_01 -> supp_02
  src/                          the Python code of each step
  tests/  slurms/tests.sh       checks on synthetic data
  data/                         results (created by the run, not in git)
  logs/<step>/                  one log per SLURM job
```

2. Install (once per cluster account)

```
git clone https://github.com/ssenci97/SCambrian.git
cd SCambrian
sbatch esmc_config.sh
tail -f logs/setup_ESMC.log
```

The job builds the conda env `./.env`: Python 3.10, PyTorch 2.4.1 cu121, flash-attn 2.6.3, ESM, Scanpy, MMseqs2, pytest. It succeeded when the log ends with `All pipeline dependencies successfully verified!`.

The **first** build also writes exact lock files into `env/`. Commit them once, so every later install is identical:

```
git add env/ && git commit -m "env: lock" && git push
```

Running `sbatch esmc_config.sh` again never changes an existing `./.env`; it only re-checks it. To use the env in a shell:

```
source ~/scratch/miniconda/etc/profile.d/conda.sh
conda activate ./.env
```

3. Check that everything works

command	runs on	time
`python -m pytest -m static`	login node	$<$ 1 min
`sbatch slurms/tests.sh`	EPYC	a few min
`sbatch --partition=DGX --gres=gpu:A100:1 slurms/tests.sh gpu`	DGX	a few min

The tests use synthetic proteins in a temporary folder and never touch your configs or results. A pass ends with `All selected tests passed` in `logs/tests/tests_<jobid>.log`.

4. Configure your analysis

`configs/query_configs.tsv`: one tab-separated row per input, with no comments.

- `label`: short name used in file names, e.g. `aegypti`
- `query`: a UniProtKB query, e.g. `(organism_id:7159)`, or the path of a local FASTA file, e.g. `raw/proteomes/koreicus_seqs.fasta`
- `exact_species`: name shown in figures, e.g. `Ae. aegypti`
- `input_type`: `target` (compared) or `probe` (functional reference)

`configs/param_configs.yaml`: one section per step. The keys you will most often change:

- `extract_reps_biohub.models`: `esmc_300m`, `esmc_600m`, `esmc_6b`
- `npz_scanpy.leiden_resolution`, `npz_scanpy.knn_neighbors`, `npz_scanpy.seed`
- `h5ad_scanpy.*`: thresholds for selecting clusters in the report

Run `python -m pytest -m static` after editing: it catches typos in keys and bad TSV rows.

5. Run

```
DRY_RUN=1 bash slurms/run_all.sh      # show what would be submitted
bash slurms/run_all.sh                # submit the whole chain
bash slurms/run_all.sh 03 supp_01     # only some steps
squeue -u $USER                       # follow the jobs; logs in logs/<step>/
```

Run these on the login node, not with `sbatch`. Each step starts only if the previous one succeeded.

step	does	output in `data/`
01	UniProt download / copy local FASTA	`sequences/`, `features/`
02	MMseqs2 dedup and clustering	`dedup/`, `clusters/`
03	ESMC embeddings (GPU)	`esm-reps/<model>/`
supp_01	Leiden clusters, UMAP, t-SNE	`scanpy/<run>/*.h5ad`
supp_02	PDF report, tables, 3D viewer	`scanpy/<run>/`

6. Results, changes, reruns

- Report: `<run>_scanpy.pdf` in `data/scanpy/<run>/`.
- Interactive viewer: `interactive/*.html` in the same folder (open it in a browser).
- Notebooks in `nbs/` load every stage; use `./.env` as the kernel.
- After changing a config, run `bash slurms/run_all.sh` again: only missing or outdated outputs are rebuilt. To force a step, set `overwrite: true` in its section. For report-only thresholds, set `h5ad_scanpy.overwrite: true`.
- Earlier results (before 2026-11-01): `git fetch --tags && git worktree add ../SCambrian-pre-reorg pre-reorg`.

Full details and troubleshooting are in `docs/USAGE.md`.

7. Coming next (not available yet): project folders

The plan separates the **tool**, installed once, from **projects**, one folder per study, anywhere on disk:

```
install once:   git clone ... && sbatch esmc_config.sh   (the tool + ./.env)
new project:    scambrian init ~/scratch/projects/my-study
```

```
first try:      scambrian init --demo ~/scratch/projects/demo
run:            cd ~/scratch/projects/my-study && scambrian run --dry-run
                scambrian run
results:        ~/scratch/projects/my-study/runs/<step>/<run_id>/
```

A project folder holds only your inputs and results: `scambrian.yaml`, `query_configs.tsv`, `inputs/`, `runs/` and `logs/`. The SLURM scripts stay inside the tool, so fixes reach every project. `--demo` creates a tiny synthetic study that runs the real SLURM chain in minutes. Each run folder will carry an RO-Crate recording the code version, config, inputs and environment.

today	after the change
edit `configs/*` in the clone	edit `scambrian.yaml` and `query_configs.tsv` in a project
`bash slurms/run_all.sh`	`scambrian run`
results in `data/`, overwritten	results in `runs/<step>/<run_id>/`, never overwritten
one analysis per clone	many projects per install